KMP Pattern Searching Algorithm in Java

KMP stands for **Knuth-Morris-Pratt**. It is a string-searching algorithm used to find a pattern inside a larger text efficiently.

It is especially useful when we need to search for a word or sequence of characters in a long string.

1. Real-life example

Suppose we have the text:

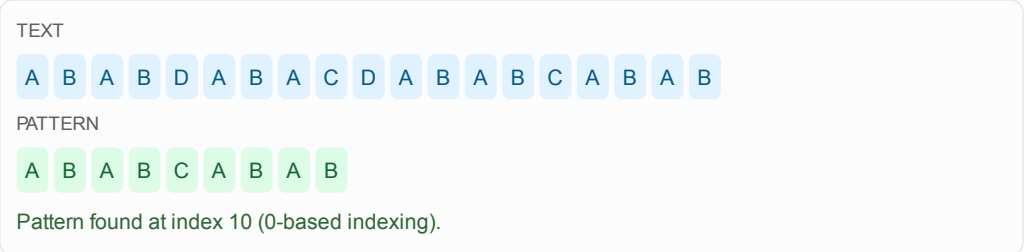
`ABABDABACDABABCABAB`

And we want to search for the pattern:

`ABABCABAB`

The KMP algorithm checks whether the pattern exists in the text. If it finds a mismatch, it uses previously matched characters to decide where to continue, rather than starting the comparison from scratch.

How pattern searching works



2. Important terms

- **Text:** The complete string in which we search.
- **Pattern:** The smaller string we want to find.
- **LPS array:** Longest Proper Prefix which is also a Suffix.
- **Mismatch:** When the current text character and pattern character are different.

What is an LPS array?

LPS is the key idea behind KMP.

A *prefix* is a sequence starting from the beginning of a string. A *suffix* is a sequence ending at the end of a string.

A *proper prefix* cannot be the entire string.

For example, consider the pattern `ABAB`.

Substring	LPS value
`A`	0
`AB`	0
`ABA`	1
`ABAB`	2

For `ABAB`, the longest proper prefix that is also a suffix is `AB`, whose length is 2.

Therefore, the LPS array is:

`[0, 0, 1, 2]`

3. Java implementation

Java

```
public class KMP {

    static void computeLPS(String pattern, int[] lps) {
        int len = 0;
        int i = 1;

        lps[0] = 0;

        while (i < pattern.length()) {
            if (pattern.charAt(i) == pattern.charAt(len)) {
                len++;
                lps[i] = len;
                i++;
            } else {
                if (len != 0) {
                    len = lps[len - 1];
                } else {
                    lps[i] = 0;
                    i++;
                }
            }
        }
    }

    static void KMPSearch(String text, String pattern) {
        int n = text.length();
        int m = pattern.length();

        int[] lps = new int[m];
        computeLPS(pattern, lps);

        int i = 0;
        int j = 0;

        while (i < n) {
            if (text.charAt(i) == pattern.charAt(j)) {
                i++;
                j++;
            }

            if (j == m) {
                System.out.println("Pattern found at index " + (i - j));
                j = lps[j - 1];
            } else if (i < n &&
                       text.charAt(i) != pattern.charAt(j)) {
                if (j != 0) {
                    j = lps[j - 1];
                } else {
                    i++;
                }
            }
        }
    }

    public static void main(String[] args) {
        String text = "ABABDABACDABABCABAB";
        String pattern = "ABABCABAB";

        KMPSearch(text, pattern);
    }
}
```

Output:

```
Pattern found at index 10
```

4. Code explanation, step by step

Part A: `computeLPS()`

This method creates the LPS array for the pattern.

- `len` stores the length of the matching prefix and suffix.
- `i` moves through the pattern, starting at index 1.

- `lps[0] = 0` because a single character has no proper prefix.
- If the characters match, increase `len` and store it in `lps[i]`.
- If they do not match, use an earlier LPS value to try a shorter prefix.
- If no prefix matches, store `0` and move forward.

Part B: `KMPSearch()`

This method searches for the pattern inside the text.

- `n` stores the text length.
- `m` stores the pattern length.
- `i` points to the current text character.
- `j` points to the current pattern character.
- When characters match, increment both `i` and `j`.
- When `j == m`, the complete pattern has been found. Its starting index is `i - j`.
- When there is a mismatch, use the LPS array to shift the pattern intelligently.
- If `j == 0`, there is no partial match to reuse, so advance `i`.

Why is this efficient? The text index never moves backward. KMP reuses previously matched information instead of repeating comparisons unnecessarily.

5. Time and space complexity

Operation	Complexity
Building the LPS array	$O(m)$
Searching the text	$O(n)$
Total time	$O(n+m)$
Extra space	$O(m)$

Here, n is the text length and m is the pattern length.

6. KMP vs. naive pattern searching

Feature	Naive algorithm	KMP algorithm
Basic idea	Try each possible starting position	Reuse matching-prefix information
Worst-case time	$O(nm)$	$O(n+m)$
Extra space	$O(1)$	$O(m)$
Best use	Small, simple searches	Longer texts and repeated character patterns

7. Quick classroom questions

```
import java.util.Scanner;

class KMP_String_Matching {

    // Method to perform KMP Search
    void KMPSearch(String pat, String txt) {
        int M = pat.length();
        int N = txt.length();

        // Create lps[] that will hold the longest prefix suffix values for the
pattern
```

Show more

KMP Pattern Searching in Java — Line-by-Line Explanation

Let's understand your code in **simple, classroom-friendly language**, step by step, with an example and sample output.

1. What does this program do?

The KMP (Knuth-Morris-Pratt) algorithm searches for a smaller string called a **pattern** inside a larger string called **text**.

For example:

- Text: `ABABDABACDABABCABAB`
- Pattern: `ABAB`

The program finds every occurrence of `ABAB` in the text and prints its starting index.

2. Understanding the complete program

Part 1: Import Scanner and declare the class

```java
import java.util.Scanner;

class KMP_String_Matching {
```

- `import java.util.Scanner;` allows us to take input from the user.
- `class KMP_String_Matching` defines a class named `KMP_String_Matching`.
- All the methods and logic of the program are written inside this class.

Part 2: The `KMPSearch()` method

```java
void KMPSearch(String pat, String txt) {
    int M = pat.length();
    int N = txt.length();
```

- `void` means the method does not return a value.
- `KMPSearch()` is the method that searches for the pattern.
- `pat` represents the pattern we want to find.
- `txt` represents the complete text where we search.
- `M` stores the length of the pattern.
- `N` stores the length of the text.

For example, if `pat = "ABAB"` and `txt = "ABABABC"`:

- `M = 4`
- `N = 7`

Part 3: Create the LPS array

```java
int[] lps = new int[M];
int j = 0;
```

- `int[] lps = new int[M];` creates an integer array to store the LPS values for the pattern.
- `j = 0` represents the current index of the pattern.

What is LPS?

LPS stands for *Longest Proper Prefix which is also a Suffix*.

It tells KMP how many characters can be reused after a mismatch, helping avoid unnecessary comparisons.

For the pattern `ABAB`, the LPS array is:

Pattern: ABAB

A	B	A	B
Index 0	Index 1	Index 2	Index 3

LPS array

0	0	1	2

For example, the substring ABAB has LPS value 2 because AB is both a proper prefix and a suffix.

Part 4: Calculate the LPS array

```java
computeLPSArray(pat, M, lps);
```

This calls another method named `computeLPSArray()`.

It calculates the LPS value for every position in the pattern and stores those values in the `lps` array.

We calculate LPS values before searching because KMP needs them to handle mismatches efficiently.

Part 5: Start searching in the text

```java
int i = 0;

while (i < N) {
```

- `i = 0` initializes the index of the text.
- `while (i < N)` repeats the search as long as `i` is within the text.

The algorithm uses two indexes:

Variable	Purpose
`i`	Current character in the text
`j`	Current character in the pattern

Part 6: Compare the characters

```java
if (pat.charAt(j) == txt.charAt(i)) {
    j++;
    i++;
}
```

- `charAt()` accesses a character at a particular index.
- If the pattern character matches the text character, both indexes move forward.
- `j++` moves to the next pattern character.
- `i++` moves to the next text character.

For example:

```
Text:    A B A B A B C
Pattern:  A B A B
```

When the characters match, the algorithm continues comparing the next characters.

Part 7: Check whether the pattern is found

```java
if (j == M) {
    System.out.println("Found pattern at index " + (i - j));
    j = lps[j - 1];
}
```

This is an important part of the algorithm.

- `j == M` means all characters of the pattern have matched.
- `i - j` calculates the starting index of the pattern in the text.
- `System.out.println()` displays the matching index.
- `j = lps[j - 1]` allows the algorithm to continue searching for overlapping matches without starting over.

Example: If `i = 4` and `j = 4`, then:

$$i - j = 4 - 4 = 0$$

So the pattern starts at index `0`.

Part 8: Handle a mismatch

```java
else if (i < N && pat.charAt(j) != txt.charAt(i)) {
```

This condition checks whether:

1. The text still has characters left.
2. The current pattern character does not match the current text character.

Now the algorithm must decide how to continue.

```java
if (j != 0)
    j = lps[j - 1];
else
    i = i + 1;
```

There are two cases.

Case 1: `j != 0`

Some characters have already matched. Instead of moving the text index backward, KMP uses the LPS array to determine where the pattern comparison should resume.

Case 2: `j == 0`

No pattern characters have matched at the current position. Therefore, the algorithm moves to the next text character using `i = i + 1`.

This is what makes KMP more efficient than the basic pattern-searching approach.

3. Understanding `computeLPSArray()`

This method builds the LPS array before the actual search begins.

```java
void computeLPSArray(String pat, int M, int[] lps) {
    int len = 0;
    int i = 1;
    lps[0] = 0;
```

- `len = 0` stores the length of the previously identified matching prefix and suffix.
- `i = 1` begins checking from the second character.
- `lps[0] = 0` because a single character cannot have a non-empty proper prefix.

Next:

```java
while (i < M) {
```

The loop calculates LPS values from index `1` through index `M - 1`.

When characters match

```java
if (pat.charAt(i) == pat.charAt(len)) {
    len++;
    lps[i] = len;
    i++;
}
```

If the characters match:

1. Increase `len`.
2. Store `len` in the current LPS position.
3. Move to the next character.

When characters do not match

```java
else {
    if (len != 0) {
        len = lps[len - 1];
    } else {
        lps[i] = len;
        i++;
    }
}
```

There are two possibilities:

- If `len != 0`, try a shorter prefix using an earlier LPS value. Notice that `i` does not move yet.
- If `len == 0`, there is no shorter non-empty prefix to try. Store `0` and move to the next character.

This process builds the LPS array without repeatedly checking every possible prefix.

4. Understanding the `main()` method

```java
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
```

- `main()` is the starting point of the Java program.
- `Scanner sc` creates an object for reading user input.

```java
System.out.print("Enter the text to search in: ");
String txt = sc.nextLine();
```

Displays a message and reads the complete text entered by the user.

Java

```
System.out.print("Enter the pattern to search for: ");
String pat = sc.nextLine();
```

Displays another message and reads the pattern to search for.

Java

```
new KMP_String_Matching().KMPSearch(pat, txt);
```

- Creates an object of the `KMP_String_Matching` class.
- Calls the `KMPSearch()` method.
- Passes the pattern first and the text second, matching the method's parameter order.

5. Dry run with an example

Let's use a small example that demonstrates overlapping matches.

Input:

```
Enter the text to search in: ABABAB
Enter the pattern to search for: ABAB
```

The pattern `ABAB` occurs twice.

Match 1 — starting index 0

A	B	A	B	A	B
0	1	2	3	4	5

ABAB matches at index 0.

Match 2 — starting index 2

A	B	A	B	A	B
0	1	2	3	4	5

ABAB matches again at index 2.

Output:

```
Found pattern at index 0
Found pattern at index 2
```

Notice that the second match overlaps the first. The LPS array allows KMP to find it without restarting the entire search.

6. Time and space complexity

Let `N` be the length of the text and `M` be the length of the pattern.

Operation	Complexity
Calculate the LPS array	$O(M)$
Search for the pattern	$O(N)$
Total time complexity	$O(N + M)$
Extra space for LPS	$O(M)$

7. Important points to remember

- KMP uses two indexes: `i` for the text and `j` for the pattern.
- The LPS array helps the algorithm avoid unnecessary comparisons.
- The text index `i` never moves backward.
- The expression `i - j` gives the starting index of a complete match.
- After finding a match, `j = lps[j - 1]` helps detect overlapping matches.

One small improvement: Your code assumes that the pattern is not empty. If the user enters an empty pattern, `pat.charAt(j)` will cause an error. You can handle this case separately before beginning the search.

ChatGPT can make mistakes. Check important info.